



动态规划补充

Dynamic Programming —— 第二弹

Froggy

2022 年 2 月 16 日

Contents

1 SOS DP

- 离线
- 半在线

2 DP 状态设计的优化

3 ex 背包

离线の SOS DP

- 说白了就是求这个玩意：

$$f_S = \sum_{T \subseteq S} a_T$$

离线の SOS DP

- 说白了就是求这个玩意：

$$f_S = \sum_{T \subseteq S} a_T$$

- 离线方法不再赘述，写个和高维前缀和一样的 dp 就行了。

「CF800D」 Varying Kibibits

- 有一个大小为 n 的序列 T 。

「CF800D」 Varying Kibibits

- 有一个大小为 n 的序列 T 。
- 对于某个非空序列 L ，定义 $f(L)$ ：将 L 中的元素十进制下的高位用 0 补全，取每一位的最小值组成一个新数，作为函数的值。

「CF800D」 Varying Kibibits

- 有一个大小为 n 的序列 T 。
- 对于某个非空序列 L ，定义 $f(L)$ ：将 L 中的元素十进制下的高位用 0 补全，取每一位的最小值组成一个新数，作为函数的值。
- 定义 $G(x) = x \left(\sum_{S \subseteq T, f(S)=x} \left(\sum_{y \in S} y \right)^2 \right) \bmod (10^9 + 7)$

「CF800D」 Varying Kibibits

- 有一个大小为 n 的序列 T 。
- 对于某个非空序列 L ，定义 $f(L)$ ：将 L 中的元素十进制下的高位用 0 补全，取每一位的最小值组成一个新数，作为函数的值。
- 定义 $G(x) = x \left(\sum_{S \subseteq T, f(S)=x} \left(\sum_{y \in S} y \right)^2 \right) \bmod (10^9 + 7)$
- 求 $G(0), G(1), \dots, G(999999)$ 。

「CF800D」 Varying Kibibits

- 在这道题目中，维度是 6，每一维大小为 10。

「CF800D」 Varying Kibibits

- 在这道题目中，维度是 6，每一维大小为 10。
- dp 的时候分别记录 0 次方和，1 次方和，2 次方和，这样就能求出 $G(x)$ 的高维后缀和。

「CF800D」 Varying Kibibits

- 在这道题目中，维度是 6，每一维大小为 10。
- dp 的时候分别记录 0 次方和，1 次方和，2 次方和，这样就能求出 $G(x)$ 的高维后缀和。
- 最后再做一遍高维差分就好了。

半在线の SOS DP

- 求类似于这个玩意：

$$f_S = H_1 \left(\sum_{T \subset S} g_T \right), g_S = H_2 \left(\sum_{T \subseteq S} f_T \right)$$

半在线の SOS DP

- 求类似于这个玩意：

$$f_S = H_1 \left(\sum_{T \subset S} g_T \right), g_S = H_2 \left(\sum_{T \subseteq S} f_T \right)$$

- 其中 H_1 和 H_2 是某种简单的函数。

半在线の SOS DP

- 由于是半在线，所以外层循环需要枚举集合，并且需要牺牲空间。

半在线の SOS DP

- 由于是半在线，所以外层循环需要枚举集合，并且需要牺牲空间。
- 分别设 $f_{S,i}$ 和 $g_{S,i}$ 分别表示只做 S 前 i 维的“前缀和”的 dp 值。

半在线の SOS DP

- 由于是半在线，所以外层循环需要枚举集合，并且需要牺牲空间。
- 分别设 $f_{S,i}$ 和 $g_{S,i}$ 分别表示只做 S 前 i 维的“前缀和”的 dp 值。
- 转移的时候先枚举 S ，然后再枚举 i 进行转移。

「CodeChef STR_FUNC」 Strange Functions

- 给一个长度为 n 的序列 a 。

「CodeChef STR_FUNC」 Strange Functions

- 给一个长度为 n 的序列 a 。

-

$$f_S = (a_S)^2 + \left(\sum_{T \subset S} g_T \right)^2, g_S = \sum_{T \subseteq S} (f_T)^2$$

「CodeChef STR_FUNC」 Strange Functions

- 给一个长度为 n 的序列 a 。

-

$$f_S = (a_S)^2 + \left(\sum_{T \subset S} g_T \right)^2, g_S = \sum_{T \subseteq S} (f_T)^2$$

- $\forall S$, 求出 f_S 和 g_S 。

「CodeChef STR_FUNC」 Strange Functions

- 给一个长度为 n 的序列 a 。

-

$$f_S = (a_S)^2 + \left(\sum_{T \subset S} g_T \right)^2, g_S = \sum_{T \subset S} (f_T)^2$$

- $\forall S$, 求出 f_S 和 g_S 。
- $1 \leq n \leq 6 \times 10^5$ 。

「CodeChef STR_FUNC」 Strange Functions

- 给一个长度为 n 的序列 a 。

-

$$f_S = (a_S)^2 + \left(\sum_{T \subset S} g_T \right)^2, g_S = \sum_{T \subseteq S} (f_T)^2$$

- $\forall S$, 求出 f_S 和 g_S 。
- $1 \leq n \leq 6 \times 10^5$ 。
- 板子题，直接照上一页说的做就好了。

Contents

1 SOS DP

2 DP 状态设计的优化

3 ex 背包

「uoj Goodbye Xinchou」 赵云八卦阵

- 给一个长度为 n 的序列 a ，你可以选择一个 i ，然后让 a_i 变成 $a_i \oplus a_{i-1}$ 。 ($\oplus$ 表示异或)

「uoj Goodbye Xinchou」 赵云八卦阵

- 给一个长度为 n 的序列 a , 你可以选择一个 i , 然后让 a_i 变成 $a_i \oplus a_{i-1}$ 。 ($\oplus$ 表示异或)
- 如果没有操作次数限制, 求这个序列最长上升子序列长度的最大值。

「uoj Goodbye Xinchou」 赵云八卦阵

- 给一个长度为 n 的序列 a ，你可以选择一个 i ，然后让 a_i 变成 $a_i \oplus a_{i-1}$ 。（ $\oplus$ 表示异或）
- 如果没有操作次数限制，求这个序列最长上升子序列长度的最大值。
- $n \leq 10^6, 1 \leq a_i < 2^{60}$ 。

「uoj Goodbye Xinchou」 赵云八卦阵

- 首先不难发现 a_i 最终可以变成 a_i 异或上 $\{a_1, \dots, a_{i-1}\}$ 的一个子集。

「uoj Goodbye Xinchou」 赵云八卦阵

- 首先不难发现 a_i 最终可以变成 a_i 异或上 $\{a_1, \dots, a_{i-1}\}$ 的一个子集。
- 常规 dp 肯定是设 $dp_{i,j}$ 表示前 i 个位置，最后一个数为 j 的 lis。

「uoj Goodbye Xinchou」 赵云八卦阵

- 首先不难发现 a_i 最终可以变成 a_i 异或上 $\{a_1, \dots, a_{i-1}\}$ 的一个子集。
- 常规 dp 肯定是设 $dp_{i,j}$ 表示前 i 个位置，最后一个数为 j 的 lis。
- 不过有个典中典技巧就是 dp 的时候某一维巨大，但是答案很小并且单调，就可以把答案和这一维 dp 状态交换。

「uoj Goodbye Xinchou」 赵云八卦阵

- 首先不难发现 a_i 最终可以变成 a_i 异或上 $\{a_1, \dots, a_{i-1}\}$ 的一个子集。
- 常规 dp 肯定是设 $dp_{i,j}$ 表示前 i 个位置，最后一个数为 j 的 lis。
- 不过有个典中典技巧就是 dp 的时候某一维巨大，但是答案很小并且单调，就可以把答案和这一维 dp 状态交换。
- 所以状态可以改成 $dp_{i,j}$ 表示前 i 个位置，长度为 j 的上升子序列末尾的位置的最小值。

「uoj Goodbye Xinchou」赵云八卦阵

- 到了这一步，不难通过构建出每个前缀的线性基，把 dp 优化到 $\mathcal{O}(n^2 \log V)$ 。

「uoj Goodbye Xinchou」赵云八卦阵

- 到了这一步，不难通过构建出每个前缀的线性基，把 dp 优化到 $\mathcal{O}(n^2 \log V)$ 。
- 又因为线性基一共只有 $\mathcal{O}(\log V)$ 种，如果 a_i 能被线性基表示出来，可以看作 $a_i = 0$ ，所以一共只有 $\mathcal{O}(\log V)$ 段，每段线性基相同。不妨把 dp 的第一维状态的含义改成“段”。

「uoj Goodbye Xinchou」赵云八卦阵

- 转移第 i 段的时候，可以算出每个 $dp_{i,j}$ 在线性基中的排名，转化为排名后用单调队列优化转移，最后在线性基中查询排名对应的值即可。

「uoj Goodbye Xinchou」赵云八卦阵

- 转移第 i 段的时候，可以算出每个 $dp_{i,j}$ 在线性基中的排名，转化为排名后用单调队列优化转移，最后在线性基中查询排名对应的值即可。
- 这么做时间复杂度是 $\mathcal{O}(n \log^2 V)$ 。

「uoj Goodbye Xinchou」赵云八卦阵

- 转移第 i 段的时候，可以算出每个 $dp_{i,j}$ 在线性基中的排名，转化为排名后用单调队列优化转移，最后在线性基中查询排名对应的值即可。
- 这么做时间复杂度是 $\mathcal{O}(n \log^2 V)$ 。
- 瓶颈在于查排名。不妨 dp 值直接去记录排名，然后去研究一波在新线性基里排名的变化。

「uoj Goodbye Xinchou」赵云八卦阵

- 转移第 i 段的时候，可以算出每个 $dp_{i,j}$ 在线性基中的排名，转化为排名后用单调队列优化转移，最后在线性基中查询排名对应的值即可。
- 这么做时间复杂度是 $\mathcal{O}(n \log^2 V)$ 。
- 瓶颈在于查排名。不妨 dp 值直接去记录排名，然后去研究一波在新线性基里排名的变化。
- 具体方法不再赘述，可以去看原题题解。

「uoj Goodbye Xinchou」赵云八卦阵

- 转移第 i 段的时候，可以算出每个 $dp_{i,j}$ 在线性基中的排名，转化为排名后用单调队列优化转移，最后在线性基中查询排名对应的值即可。
- 这么做时间复杂度是 $\mathcal{O}(n \log^2 V)$ 。
- 瓶颈在于查排名。不妨 dp 值直接去记录排名，然后去研究一波在新线性基里排名的变化。
- 具体方法不再赘述，可以去看原题题解。
- $\mathcal{O}(n \log V)$ 。

「Huawei Camp Day1」 Survey

- 给出 n 和 m 和一个长度为 n 的非负整数序列 a , 你可以找一个长度为 n 的非负序列 x , 满足 $\sum x_i \leq m$ 。

「Huawei Camp Day1」 Survey

- 给出 n 和 m 和一个长度为 n 的非负整数序列 a , 你可以找一个长度为 n 的非负序列 x , 满足 $\sum x_i \leq m$.
- 请你求出 $\sum_{i=1}^n \sum_{j=1}^n [x_i \geq a_j]$ 的最大值。

「Huawei Camp Day1」 Survey

- 给出 n 和 m 和一个长度为 n 的非负整数序列 a , 你可以找一个长度为 n 的非负序列 x , 满足 $\sum x_i \leq m$.
- 请你求出 $\sum_{i=1}^n \sum_{j=1}^n [x_i \geq a_j]$ 的最大值。
- $1 \leq n \leq 10^3, 1 \leq m \leq 5 \times 10^3$.

「Huawei Camp Day1」 Survey

- 枚举物品大小然后枚举个数，这样背包的复杂度肯定寄了。

「Huawei Camp Day1」 Survey

- 枚举物品大小然后枚举个数，这样背包的复杂度肯定寄了。
- 既然物品个数有限制但是大小没限制，可以尝试把 dp 状态设成：
 $dp_{i,j,k}$ 表示 $\geq i$ 的有 k 个，目前的和为 j 的答案。

「Huawei Camp Day1」 Survey

- 枚举物品大小然后枚举个数，这样背包的复杂度肯定寄了。
- 既然物品个数有限制但是大小没限制，可以尝试把 dp 状态设成： $dp_{i,j,k}$ 表示 $\geq i$ 的有 k 个，目前的和为 j 的答案。
- 由于 $k \leq \lfloor \frac{j}{i} \rfloor$ ，所以后缀和优化以后复杂度是 $\mathcal{O}(nm \log m)$ 的。

Contents

1 SOS DP

2 DP 状态设计的优化

3 ex 背包

- 子集和
- $(\max, +)$ 卷积

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。
- 你需要对于 $\forall k \in [0, C]$ ，求出是否存在一个物品子集重量等于 k 。

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。
- 你需要对于 $\forall k \in [0, C]$ ，求出是否存在一个物品子集重量等于 k 。
- 经典问题，模拟赛出过无数次，但是好像都把复杂度搞错了。

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。
- 你需要对于 $\forall k \in [0, C]$ ，求出是否存在一个物品子集重量等于 k 。
- 经典问题，模拟赛出过无数次，但是好像都把复杂度搞错了。
- 对相同重量的物品二进制分组，然后用 bitset 优化。

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。
- 你需要对于 $\forall k \in [0, C]$ ，求出是否存在一个物品子集重量等于 k 。
- 经典问题，模拟赛出过无数次，但是好像都把复杂度搞错了。
- 对相同重量的物品二进制分组，然后用 bitset 优化。
- 时间复杂度是 $\mathcal{O}(\frac{C\sqrt{C}}{\omega})$ 。

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。
- 你需要对于 $\forall k \in [0, C]$ ，求出是否存在一个物品子集重量等于 k 。
- 经典问题，模拟赛出过无数次，但是好像都把复杂度搞错了。
- 对相同重量的物品二进制分组，然后用 bitset 优化。
- 时间复杂度是 $\mathcal{O}(\frac{C\sqrt{C}}{\omega})$ 。
- 证明：设 $\sum w_i a_i = C$ ，那么物品个数是 $\sum_k \sum_i [a_i \geq 2^k] \leq \sum_k \sqrt{C \times 2^{-k}}$ 。

总重量不太大

- 有 n 个物品，重量分别 w_i ，且 $\sum w_i = C$ 。
- 你需要对于 $\forall k \in [0, C]$ ，求出是否存在一个物品子集重量等于 k 。
- 经典问题，模拟赛出过无数次，但是好像都把复杂度搞错了。
- 对相同重量的物品二进制分组，然后用 bitset 优化。
- 时间复杂度是 $\mathcal{O}(\frac{C\sqrt{C}}{\omega})$ 。
- 证明：设 $\sum w_i a_i = C$ ，那么物品个数是 $\sum_k \sum_i [a_i \geq 2^k] \leq \sum_k \sqrt{C \times 2^{-k}}$ 。
- 这玩意显然不超过 $\mathcal{O}(\sqrt{C})$ 。

单个重量不太大

- 有 n 个物品，重量分别 w_i ，且 $w_i \leq m$ 。

单个重量不太大

- 有 n 个物品，重量分别 w_i ，且 $w_i \leq m$ 。
- 你要求出是否存在一个物品子集重量等于 C 。

单个重量不太大

- 有 n 个物品，重量分别 w_i ，且 $w_i \leq m$ 。
- 你需要求出是否存在一个物品子集重量等于 C 。
- 由于 C 很大，所以考虑先找到最大的 k 满足 $\sum_{i=1}^k w_i \leq C$ ，问题就变成了从 $\{-w_1, \dots, -w_k, w_{k+1}, \dots, w_n\}$ 选子集，使得和为 $C - \sum_{i=1}^k w_i$ 。显然 $C - \sum_{i=1}^k w_i \leq m$ 。

单个重量不太大

- 有 $\mathcal{O}(nm)$ 做法，这里略。

单个重量不太大

- 有 $\mathcal{O}(nm)$ 做法，这里略。
- 更无脑的做方法就是随机打乱 w ，然后用 `bitset` 优化做一个背包。过程中期望达到的最值是 $\mathcal{O}(\sqrt{nm})$ 。

单个重量不太大

- 有 $\mathcal{O}(nm)$ 做法，这里略。
- 更无脑的做方法就是随机打乱 w ，然后用 `bitset` 优化做一个背包。过程中期望达到的最值是 $\mathcal{O}(\sqrt{nm})$ 。
- 总时间复杂度就是 $\mathcal{O}(\frac{n\sqrt{nm}}{\omega})$ 。

单个重量不太大

- 有 $\mathcal{O}(nm)$ 做法，这里略。
- 更无脑的做方法就是随机打乱 w ，然后用 bitset 优化做一个背包。过程中期望达到的最值是 $\mathcal{O}(\sqrt{nm})$ 。
- 总时间复杂度就是 $\mathcal{O}(\frac{n\sqrt{nm}}{\omega})$ 。
- 例题：「ABC221」G - Jumping sequence

其中一个凸的

- 如果俩都是凸的，双指针就好了。

其中一个凸的

- 如果俩都是凸的，双指针就好了。
- 假设序列 A 不是凸的， B 是凸的，求 $C = A + B$ ，这里的 $+$ 是 $(\max, +)$ 卷积。

其中一个凸的

- 如果俩都是凸的，双指针就好了。
- 假设序列 A 不是凸的， B 是凸的，求 $C = A + B$ ，这里的 $+$ 是 $(\max, +)$ 卷积。
- C_i 在 B 中决策点单调。直接二分栈解决就好了。

其中一个凸的

- 如果俩都是凸的，双指针就好了。
- 假设序列 A 不是凸的， B 是凸的，求 $C = A + B$ ，这里的 $+$ 是 $(\max, +)$ 卷积。
- C_i 在 B 中决策点单调。直接二分栈解决就好了。
- $\mathcal{O}(n \log n)$ 。闲得蛋疼用 ~~SMAWK~~ 就 $\mathcal{O}(n)$ 了。

重量种类少的 01 背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。不同的 w_i 有 m 个。求大小为 C 的 01 背包。

重量种类少的 01 背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。不同的 w_i 有 m 个。求大小为 C 的 01 背包。
- 对于 w_i 相同的物品，肯定按 v_i 从大到小选，是个凸函数。所以把每种 w_i 都做 (max, +) 卷积即可。

重量种类少的 01 背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。不同的 w_i 有 m 个。求大小为 C 的 01 背包。
- 对于 w_i 相同的物品，肯定按 v_i 从大到小选，是个凸函数。所以把每种 w_i 都做 (max, +) 卷积即可。
- $\mathcal{O}(n \log n + mC \log C)$ 。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。
- $\forall i > D, dp_i = \max_{x+y=i} \{dp_x + dp_y\}$ 。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。
- $\forall i > D, dp_i = \max_{x+y=i} \{dp_x + dp_y\}$ 。
- 显然一定存在一个决策点满足 $|j - k| \leq m$ 。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。
- $\forall i > D, dp_i = \max_{x+y=i} \{dp_x + dp_y\}$ 。
- 显然一定存在一个决策点满足 $|j - k| \leq m$ 。
- 如果知道了 $dp_{k-m, \dots, k+m}$ ，就可以用 (max, +) 卷积求出 $dp_{2k-m, \dots, 2k+m}$ 。于是可以倍增求解。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。
- $\forall i > D, dp_i = \max_{x+y=i} \{dp_x + dp_y\}$ 。
- 显然一定存在一个决策点满足 $|j - k| \leq m$ 。
- 如果知道了 $dp_{k-m, \dots, k+m}$ ，就可以用 (max, +) 卷积求出 $dp_{2k-m, \dots, 2k+m}$ 。于是可以倍增求解。
- 初始化时，暴力计算 $dp_{0, \dots, 2m}$ 。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。
- $\forall i > D, dp_i = \max_{x+y=i} \{dp_x + dp_y\}$ 。
- 显然一定存在一个决策点满足 $|j - k| \leq m$ 。
- 如果知道了 $dp_{k-m, \dots, k+m}$ ，就可以用 (max, +) 卷积求出 $dp_{2k-m, \dots, 2k+m}$ 。于是可以倍增求解。
- 初始化时，暴力计算 $dp_{0, \dots, 2m}$ 。
- $\mathcal{O}(n \log n + m^2 \log C)$ 。

重量小的完全背包

- 有 n 个物品，每个物品有重量 w_i ，价值 v_i 。 $w_i \leq m$ 。求价值最大的可重子集满足重量和不超过 C 。
- $\forall i > D, dp_i = \max_{x+y=i} \{dp_x + dp_y\}$ 。
- 显然一定存在一个决策点满足 $|j - k| \leq m$ 。
- 如果知道了 $dp_{k-m, \dots, k+m}$ ，就可以用 (max, +) 卷积求出 $dp_{2k-m, \dots, 2k+m}$ 。于是可以倍增求解。
- 初始化时，暴力计算 $dp_{0, \dots, 2m}$ 。
- $\mathcal{O}(n \log n + m^2 \log C)$ 。
- 板子题：「gym 101064」The Knapsack problem